School of Electrical Engineering
University of Belgrade



Marko Dasic

Sentiment Analysis of Movie Reviews Using Machine Learning Methods

Mentor: Assoc. Prof. Vladimir Jocovic, PhD

Belgrade, 2026.

Contents

1. Introduction 1
2. Overview of Existing Solutions and Tools 2
2.1. Sentiment Analysis Approaches 2
2.2. Vector Representation of Text – TF-IDF 2
2.3. Machine Learning Models Used in the Experiment 3
2.3.1. Naive Bayes 3
2.3.2. Logistic Regression 3
2.3.3. Decision Tree 4
2.3.4. Random Forest 4
2.3.5. K-Nearest Neighbors (KNN) 4
2.3.6. Support Vector Machine (SVM) 4
2.4. Tools and Libraries Used in the Work 4
3. Experimental Methodology 6
3.1. Dataset and Its Characteristics 6
3.2. Text Preprocessing 8
3.3. Text Vectorization (TF-IDF) 10
3.4. Overview of the Applied Models and Common Training Workflow 12
3.4.1. Lite Variants of the Random Forest and KNN Models 13
3.5. Metrics and Evaluation method 13
3.6. Limitations of the Experiment 14
4. Results and Analysis 15
4.1. Overview of All Results 15
4.2. Naive Bayes 16
4.3. Logistic Regression 17
4.4. Decision Tree 18
4.5. Random Forest 19
4.5.1. Heavy Variant 19

4.5.2. Lite Variant 20
4.6. K-Nearest Neighbors (KNN) 21
4.6.1. Heavy Variant 22
4.6.2. Lite Variant 22
4.7. Support Vector Machine (SVM) 24
4.8. Evaluation and Analysis of Results 25
4.8.1. Influence of the Text Preprocessing Variant 25
4.8.2. Statistical Significance and Reliability of Results 26
4.8.3. Training Time and Model Size 28
4.8.4. Pareto Analysis (F1 versus “Cost”) 29
4.8.5. Precision and Recall Asymmetry 30
5. Conclusion 31
5.1. Limitations of the Conducted Experiment 31
5.2. Implementation Challenges 32
5.3. Directions for Further Work 32

1. Introduction

With the rapid growth in the amount of textual content that users publish online every day, in the form of product, movie, or service reviews or comments on social networks, there arose a need for automated methods that can assess whether a given text expresses a positive or negative attitude of the author. Manual analysis of such a large volume of text is not practically feasible, so the task of recognizing sentiment in text is addressed by applying machine learning methods to appropriately represented data.

This thesis deals with sentiment analysis of movie reviews, i.e. the problem of binary text classification in which a movie review is assigned to one of two categories: positive or negative. For the purposes of the experiment, the publicly available IMDB Dataset of 50K Movie Reviews was used, containing 50,000 movie reviews, evenly distributed between the positive and negative classes.

The aim of the work is to apply and compare several classical machine learning algorithms on the same dataset, using the same numerical representation of text (TF-IDF), in order to compare their performance and determine which model achieves the best results under the conditions of the conducted experiment. The models considered are: Naive Bayes, Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM). The aim of the work is also to examine the influence of different text preprocessing techniques (without normalization, stemming, lemmatization, and more aggressive stemming) on the performance of each of the listed models.

As part of the experiment, the review text was preprocessed by removing HTML tags, punctuation, and numbers, while retaining negation words in order to preserve the sentiment signal they carry. The processed text was then represented using the TF-IDF (Term Frequency – Inverse Document Frequency) vectorization technique, while model hyperparameters were tuned using GridSearchCV. The procedure used to conduct the experiment, the evaluation metrics, and other methodological issues are described in detail in Chapter 3 of this work.

The experimental results show that the linear models, Logistic Regression and SVM, applied to the unprocessed (baseline) text, achieve the best and mutually comparable results, while more complex models such as Random Forest and KNN do not improve performance despite their considerably higher computational and memory cost. For practical applicability, lite variants of the Random Forest and KNN models were additionally created, with substantially smaller size and negligible impact on, or improvement of, the achieved results. Based on the best model, a graphical application was also developed that allows the user to enter an arbitrary review and obtain the predicted sentiment.

Sentiment analysis today represents an important tool in numerous areas, from monitoring user satisfaction with products and services, through analysis of public opinion on social networks, to content recommendation based on the impressions of other users. Comparing classical machine learning methods on a specific, well-known dataset provides insight into their advantages and limitations, and represents a basis for further research or the application of more sophisticated approaches, such as deep neural networks and transformer models.

The work is organized into five chapters. Following this introductory chapter, Chapter 2 provides an overview of the tools and libraries used in the experiment. Chapter 3 describes the methodology of the conducted experiment, including data preprocessing, vectorization, and the method of model evaluation. Chapter 4 presents and analyzes the obtained results for each of the models used, while Chapter 5, the concluding chapter, summarizes the main results of the work, the limitations of the conducted experiment, and possible directions for further research.

2. Overview of Existing Solutions and Tools

This chapter provides an overview of the approaches, methods, and tools that form the basis of the experimental part of the work. First, the basic approaches to sentiment analysis and the role of machine learning in solving this problem are presented. Then, the method of representing textual data in numerical form is described, with particular emphasis on the TF-IDF representation used in the experiment. After that, the machine learning algorithms applied to classify reviews are presented, while the programming language, libraries, and tools used to implement the experiment and develop the graphical application are described at the end of the chapter.

2.1. Sentiment Analysis Approaches

Sentiment analysis, also known as opinion mining, is a subdiscipline of natural language processing whose goal is the automatic recognition of a subjective attitude, emotion, or opinion expressed in text. In the literature, different approaches to this problem can be identified, among which the lexicon-based approach and the machine learning-based approach have traditionally been distinguished.

The first approach, lexicon-based, is based on predefined dictionaries in which each word or expression is assigned a value indicating whether that word carries a positive or negative connotation (e.g. “excellent” carries a strongly positive value, while “terrible” carries a strongly negative value). The sentiment of the entire text is then calculated by aggregating (most commonly by summing or averaging) the values of all recognized words. The advantage of this approach is that it does not require labeled data for training, but its main disadvantage is that it has difficulty capturing context, negation, sarcasm, and domain-specific word usage unless the lexicon is manually adapted to the domain.

The second approach, based on machine learning, treats sentiment analysis as a supervised binary or multiclass text classification problem: the model is trained on a set of previously labeled examples (in this work, positive and negative movie reviews), learns statistical patterns that connect text features with the corresponding sentiment class, and then applies the learned knowledge to new, unseen examples. The approach used in this work makes it possible to learn patterns characteristic of a specific problem and domain from labeled examples, at the cost of requiring a labeled dataset and computational resources for training.

In addition to these two classical approaches, in the last decade, particularly due to deep neural networks and transformer architectures (e.g. BERT), approaches based on deep learning have also been developed, achieving better results than classical machine learning methods, but at a significantly higher computational and memory cost and with lower interpretability of the resulting models. Since the focus of this work is the comparison of classical, well-established machine learning methods, approaches based on deep neural networks were not considered in the experiment, but they are listed as a possible direction for further research in the conclusion of the work.

2.2. Vector Representation of Text – TF-IDF

In order for the review text to be used as input to machine learning algorithms, it first needs to be converted into numerical form. This work uses the TF-IDF (Term Frequency – Inverse Document Frequency) technique, one of the most widespread and longest-used techniques for text vectorization in natural language processing [1]. TF-IDF assigns each term in a document a weight proportional to

the frequency with which the term appears in that document (TF), and inversely proportional to the number of documents in the corpus in which the term appears (IDF). In this way, terms that are frequent in a particular document but relatively rare in the entire corpus receive a higher weight, while terms that appear in a large number of documents and therefore have lower discriminative value receive a lower weight. The work uses the TfidfVectorizer implementation from the scikit-learn library [2], which allows direct configuration of parameters such as the n-gram range and minimum and maximum term frequencies.

An alternative to the TF-IDF representation is provided by techniques based on dense vector embeddings, such as Word2Vec, GloVe, or contextual embeddings obtained from transformer models. These techniques represent words as dense vectors in a continuous space, where semantically similar terms have similar vectors, allowing them to capture semantic relationships that TF-IDF, as a representation based on term occurrence frequency, cannot recognize. Although potentially more powerful, these techniques introduce additional complexity (the need for pretrained models or for training embeddings from scratch) that exceeds the scope of this work, so TF-IDF represents a practical and well-documented choice for comparing classical machine learning models.

2.3. Machine Learning Models Used in the Experiment

For the purposes of the experiment, six machine learning algorithms were selected that represent different families of classification approaches (probabilistic, linear, decision tree, ensemble, and distance-based methods), and all were used through their implementations in the scikit-learn library [2].

2.3.1. Naive Bayes

A probabilistic classifier based on Bayes' theorem, with the naive assumption of conditional independence of features (terms) given the class. Although this assumption is almost never true for natural language, Naive Bayes is known as a surprisingly efficient and fast text classifier, especially when combined with frequency-based representations such as TF-IDF [3]. The MultinomialNB variant was used in the work, suitable for working with non-negative feature values such as those obtained by the TF-IDF representation of text.

2.3.2. Logistic Regression

A linear model that uses the sigmoid function to convert a weighted sum of features into the probability of belonging to the positive class. The model is trained by minimizing the cross-entropy loss function (log-loss), while regularization (L1 or L2) limits model complexity and reduces the risk of overfitting when the number of features is large relative to the number of examples, which is common with TF-IDF text representations [2].

2.3.3. Decision Tree

A model that makes predictions through a sequence of successive decisions over individual features, where at each node a feature and threshold are selected that best separate the classes (most commonly based on the Gini impurity index). The advantage of this model is the direct interpretability of the resulting decision structure, while its main disadvantage is its tendency to overfit, especially in high-dimensional feature spaces such as those produced by TF-IDF [4].

2.3.4. Random Forest

An ensemble method that combines a large number of independent decision trees, where individual trees are trained on different bootstrap samples, while randomly selected subsets of features are considered when splitting nodes, and their predictions are then combined by majority voting. This reduces variance compared with an individual tree and generally produces a more stable and accurate model, at the cost of losing direct interpretability and incurring substantially higher computational and memory costs [5].

2.3.5. K-Nearest Neighbors (KNN)

A “lazy” model that does not learn an explicit decision boundary, but instead stores the entire training set and, for a new example, searches for the k most similar examples from the training set, making a prediction based on their majority class. Since KNN directly depends on a meaningful measurement of distance between examples, it is sensitive to the “curse of dimensionality” in the original, high-dimensional TF-IDF space, which in this work requires prior dimensionality reduction [2].

2.3.6. Support Vector Machine (SVM)

A model that seeks a hyperplane (decision boundary) that separates two classes with the maximum margin, i.e. the maximum distance to the nearest examples of both classes. For high-dimensional spaces such as those produced by the TF-IDF representation, the linear variant is particularly suitable, especially the linear variant (LinearSVC), which is computationally much more efficient than a classical SVM with kernel functions [6].

A more detailed theoretical presentation of each of the listed models, including the training method and selected hyperparameters, is given in Chapter 3, in the context of the specific application to the dataset used in this work.

2.4. Tools and Libraries Used in the Work

The complete experiment was implemented in the Python programming language, in the Jupyter Notebook environment, which allows executable code, textual explanations, and graphical displays to be combined in a single document, thereby achieving clarity and easier tracking of the workflow.

The pandas library was used for working with tabular data, while NumPy and SciPy were used for numerical operations on arrays and sparse matrices, with SciPy being particularly important because it enables efficient storage of TF-IDF matrices in sparse form, which, despite their large dimensions, contain a very small percentage of non-zero values (sparsity above 99% in this work).

The NLTK (Natural Language Toolkit) library [7] was used for text preprocessing, providing the standard English stop-word list, the WordNet lexical database required for lemmatization, the Snowball and Lancaster stemmers, as well as a pretrained POS (part-of-speech) tagging model, necessary for lemmatization to correctly determine the base form of a word depending on its part of speech (verb, noun, adjective).

All machine learning models used, as well as tools for vectorization (TF-IDF), hyperparameter search (GridSearchCV), and calculation of evaluation metrics, were taken from the scikit-learn library [2], one of the most widely used and best-documented Python libraries for classical machine learning. The joblib library was used to store and load trained models and vectorizers, and is suitable for objects containing large NumPy arrays.

The matplotlib and seaborn libraries were used for graphical displays (distributions, confusion matrices, and result comparisons).

Finally, for practical application of the best obtained model, a standalone graphical application was developed using the tkinter library, the standard Python library for creating graphical user interfaces, which was then packaged into a standalone executable (.exe) file using the PyInstaller tool, for easier use without the need to install a Python environment.

3. Experimental Methodology

This chapter describes the procedure for conducting the experiment, from preparing the dataset and preprocessing the textual reviews, through their numerical representation, to training and evaluating the machine learning models. The aim of the methodology is to provide conditions in which different models can be compared with one another, as well as to examine the influence of different text preprocessing methods on their performance. All models were applied to the same set of reviews and the same training and test examples, while differences in the processing procedure were controlled through four defined variants.

When defining the methodology, care was taken to ensure that all models were tested under as uniform conditions as possible. Therefore, the same training and test examples, the same basic TF-IDF representation, and the same evaluation criteria were used for all variants, while the text preprocessing method and classification algorithm were changed in a controlled manner. In this way, the obtained performance differences can be analyzed in relation to specific changes in the processing procedure and model selection, with the limitations of the experimental setup discussed in more detail at the end of the chapter.

The experiment begins with the analysis and preparation of the dataset, examining its basic characteristics, class distribution, review length, and the presence of duplicates and HTML content. After that, the textual data undergo a common cleaning procedure, after which four text variants are formed: baseline, stemming, lemmatization, and Lancaster stemming. Special attention is paid to preserving words that carry negation information, since removing them can significantly change the meaning and sentiment value of individual expressions.

To convert the processed texts into a numerical representation, TF-IDF is used, with a separate vectorizer created for each preprocessing variant. Vectorization is organized so that the parameters required to construct the representation are determined exclusively on the training set, while the test set is transformed using the already constructed vectorizer. This prevents the use of information from the test set during the training process and ensures correct model evaluation.

After vectorization, each of the four text variants is applied to the six selected machine learning algorithms. GridSearchCV with five-fold cross-validation on the training set is used to tune their hyperparameters, while the final evaluation is performed on the held-out test set. Model performance is analyzed using multiple metrics, with the F1 measure as the primary criterion for selecting the best hyperparameter combination. In addition to the basic metrics, additional statistical checks were conducted so that the obtained differences between closely ranked models could be interpreted in the context of the limitations of the experiment.

At the end of the chapter, the limitations of the methodology used are listed, primarily the fact that the main results were obtained on one fixed split of the dataset into training and test parts, and that the GridSearchCV searches did not cover the entire space of possible hyperparameters. These limitations are important for the correct interpretation of the results presented in the following chapter.

3.1. Dataset and Its Characteristics

For the purposes of the experiment, the publicly available IMDB Dataset of 50K Movie Reviews was used, downloaded from the Kaggle platform. The dataset contains a total of 50,000 movie reviews, with each review assigned a binary sentiment label: positive or negative.

Before text preprocessing was undertaken, exploratory data analysis (EDA) was conducted, with the aim of determining the basic characteristics of the dataset that directly affect decisions made later in the work. The key findings of the exploratory analysis are as follows:

- The dataset is completely balanced – 25,000 positive and 25,000 negative reviews, thereby avoiding the problem of imbalanced classes and facilitating interpretation of the evaluation metrics.
- The length of the reviews varies significantly (on average about 1,309 characters, or 231 words per review), with the presence of particularly long reviews that raise the average above the median (970 characters, or 173 words).
- There was no statistically significant difference in review length between the positive and negative classes.
- The dataset contained 418 duplicate rows, which were removed before further processing to prevent the same reviews from leaking into the training and test sets (data leakage).
- HTML tags were found in 29,200 of 50,000 reviews, remaining from the original IMDb page, which directly determined the need to remove them during preprocessing.

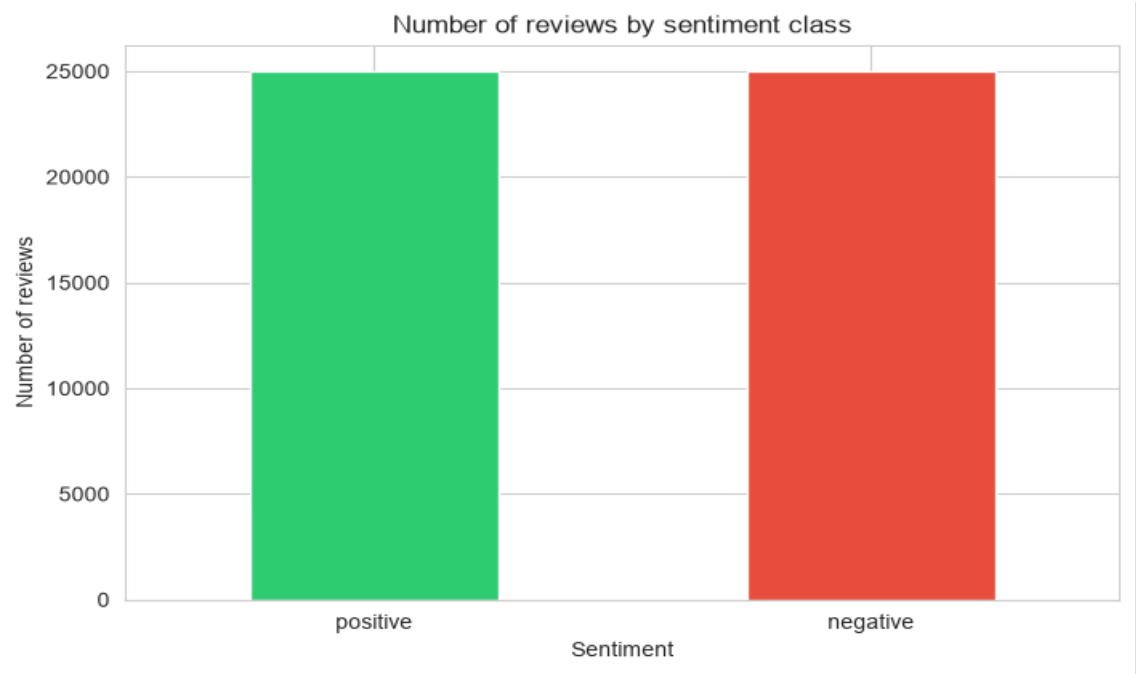


Figure Distribution of the number of positive and negative reviews

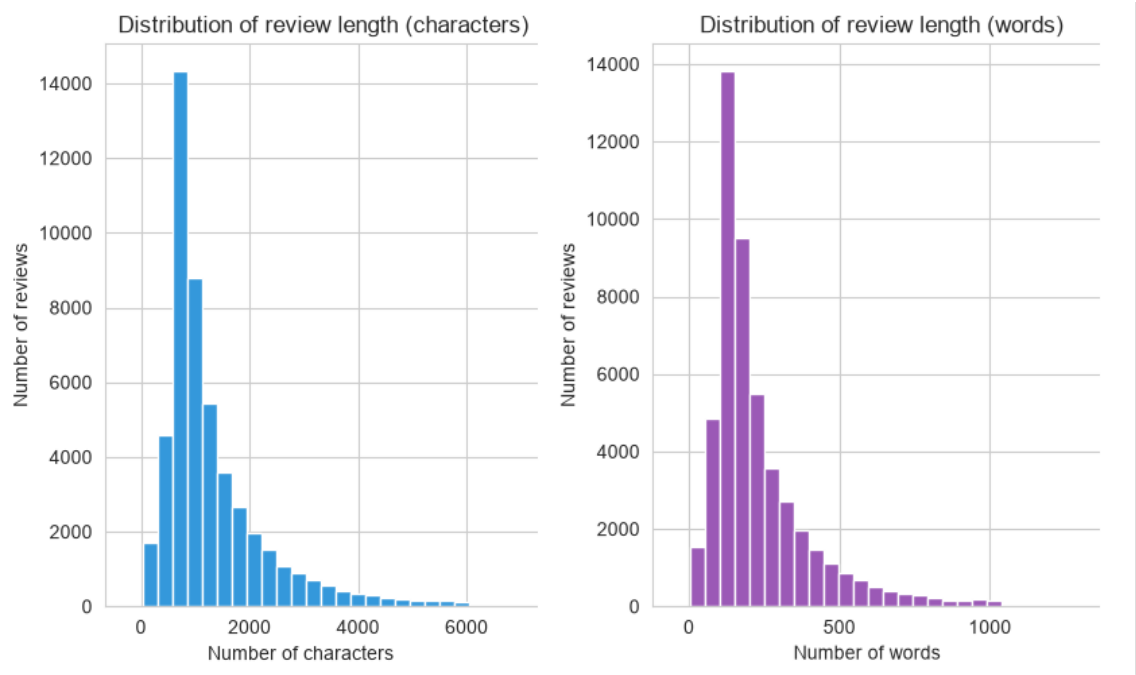


Figure Distribution of review length by number of characters and words

3.2. Text Preprocessing

Text preprocessing was performed within the common basic_clean() function, defined in the src/preprocessing.py module, so that an identical cleaning procedure was applied before each of the four comparative text normalization variants.

The cleaning cycle comprises seven steps, in a strictly defined order, where the order is not arbitrary but directly dictated by dependencies between the steps (e.g. HTML entity decoding must precede contraction recognition because the HTML-encoded apostrophe ' must first be converted into a real apostrophe):

1. Removing HTML tags and decoding HTML entities,
2. Converting the text to lowercase,
3. Expanding contractions (e.g. don't → do not, including forms without apostrophes such as isnt → is not),
4. Removing numbers and punctuation,
5. Tokenization (splitting text by spaces),
6. Removing stop words, using a customized list that preserves negation words,
7. Removing tokens shorter than two characters.

Special attention was paid to the third and sixth steps, since they directly affect preservation of the sentiment signal:

Customized stop-word list – The standard NLTK English stop-word list was reduced by removing the set of negation words (not, no, nor, never, neither), because removing them would cause phrases

such as “not good” and “good” to become identical after cleaning, thereby losing the completely opposite sentiment significance of these two phrases.

```
NEGATION_WORDS = {"not", "no", "nor", "never", "neither"}

def build_stopwords() -> set[str]:
    base = set(nltk_stopwords.words("english"))
    return base.difference(NEGATION_WORDS)
```

Source Code Segment Customized Stop-Word List

Expanding contractions – Since punctuation (including the apostrophe) is removed in the next step, contractions must be expanded beforehand; otherwise, “don't” would become the meaningless “dont” after the apostrophe is removed (or, depending on the order, would lose its negation). In addition to standard contractions with apostrophes, common informal variants without apostrophes (dont, isnt, wasnt...), common in reviews written in a casual register, as well as several informal expressions (gonna, kinda), were taken into account.

After the cleaning is completed, the resulting tokens are further branched into four parallel variants, which are tested comparatively across all selected models:

- **Baseline** – without any normalization of word forms; the control variant serving as a basis for comparison with the other variants.
- **Stemming** – crude reduction of words to their stem using the Snowball (Porter2) stemmer.
- **Lemmatization** – reduction of words to their grammatically correct base form (lemma) using the WordNet lemmatizer, with prior POS tagging of tokens.
- **Lancaster stemming** – a more aggressive stemmer than Snowball, introduced to determine whether more aggressive removal of suffixes provides additional performance improvement compared with the moderate Snowball approach.

The execution order of these four variants was chosen deliberately: the initial plan included only stemming and lemmatization, and their comparison showed that moderate (Snowball) stemming produced better results than lemmatization. Based on this finding, the baseline variant was subsequently added (to check whether normalization contributes at all to model performance compared with completely unnormalized but cleaned text) and the Lancaster variant (to check whether the trend “lower grammatical precision → better result” continues with more aggressive stemming).

The split into training and test sets was performed before branching into the four variants, so that all variants share the same reviews in the training and test sets – if the split had been performed separately for each variant, the comparison of results in Chapter 4 would be unreliable, because the differences would partly arise from different test-set content rather than exclusively from the way the text was processed. train_test_split from the scikit-learn library was used, with an 80:20 ratio, stratification by the target variable (stratify=df["sentiment"]) and a fixed random_state=1, for reproducibility of results.

The selected training/test split represents a fixed experimental setup used for all subsequent combinations of models and preprocessing variants. In this way, each model is evaluated on the same test examples, allowing direct comparison of their results. The stratified split ensures that the ratio of positive and negative reviews is preserved in both the training and test sets.

```
train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    stratify=df["sentiment"],
    random_state=1,
)
```

Source Code Segment Train/Test Split

Table Train/Test split

Dataset	Number of rows	Positive	Negative
Train	39665	50.19%	49.81%
Test	9917	50.19%	49.81%

Choosing one fixed split also has a methodological limitation: the obtained results depend on the specific examples selected for the test set. A different split of the same dataset could lead to small differences in measured performance. Nevertheless, within this work the fixed split was chosen for practical feasibility of the experiment and to ensure direct comparability of all tested combinations. Additional checks of the stability and significance of differences between models are described in Section 3.5, while the limitations of this experimental setup are discussed in more detail in Section 3.6.

3.3. Text Vectorization (TF-IDF)

The processed text, in each of the four variants, needs to be converted into a numerical form suitable for machine learning algorithms. The work uses TF-IDF representation, applying the TfidfVectorizer class from the scikit-learn library, fitted separately for each of the four preprocessing variants.

The key decisions in vectorization are as follows:

- **ngram_range=(1, 2)** – the vocabulary includes both individual words (unigrams) and pairs of consecutive words (bigrams). This is particularly important for negation phrases (“not good”, “not recommend”), which, when considered only through unigrams, would lose information about which word the negation refers to.
- **Fitting** exclusively on the training set, transforming the test set – the vocabulary and IDF weights are calculated exclusively from the training set (fit_transform), while the test set is only transformed (transform) using the existing vocabulary, thereby preventing information leakage from the test set into the training process.
- **min_df=10** – terms that occur in fewer than 10 reviews are omitted from the vocabulary, reducing the influence of extremely rare terms and potential typographical errors.
- **max_df=0.85** – terms present in more than 85% of reviews are omitted as too generic to carry discriminative information. Before applying this threshold, it was explicitly checked that the

word “not” (present in 82.4% of reviews in the training set) remains below the adopted limit, confirming that the negation will not be unjustifiably removed.

```
vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=10,
    max_df=0.85,
)
X_train = vectorizer.fit_transform(train_texts)
X_test = vectorizer.transform(test_texts)
```

Source Code Segment Key Decisions in Vectorization

The values min_df=10 and max_df=0.85 were not adopted arbitrarily, but after a diagnostic check: the first tested combination (min_df=5, max_df=0.9) resulted in a vocabulary of over 150,000 terms, after which the thresholds were tightened and confirmed by checking the actual numbers (frequency of negations and negation bigrams, number of terms eliminated by reason for elimination).

The resulting TF-IDF matrices were stored in sparse .npz format (scipy.sparse), given that matrix sparsity exceeds 99.8% in all variants – a matrix without sparse format, at dimensions on the order of 40,000×65,000, would require an impractically large amount of memory. Fitted vectorizer objects were saved in .joblib format for subsequent reuse (e.g. for prediction on a new review within the GUI application).

Sentiment labels were encoded in binary form (positive → 1, negative → 0), thereby formulating the task as standard binary classification.

The resulting vocabulary sizes by variant are summarized in the following table:

Table Overview of vocabulary size by variant

Variant	Vocabulary size (after filtering)	Unigrams	Bigrams
Baseline	64.803	22.855	41.948
Stemming	66.682	15.566	51.116
Lemmatization	66.834	18.751	48.083
Lancaster	68.378	13.094	55.284

The table shows that increasing the aggressiveness of text normalization changes the structure of the resulting vocabulary. In the baseline variant, a large part of the vocabulary consists of unigrams, while in stemming, lemmatization, and especially Lancaster stemming, the number of unigrams decreases while the number of bigrams increases. The reason for this phenomenon is that stricter normalization merges different forms of the same word into a common form, thereby reducing the number of unique individual terms. On the other hand, bigrams represent combinations of two consecutive words, so their diversity depends not only on individual word forms but also on their

context. Therefore, normalization that reduces the number of unigrams does not lead to an equally large reduction in the number of distinct bigrams. The most pronounced example is the Lancaster variant, where the number of unigrams is the smallest (13,094), while the number of bigrams is the largest (55,284), indicating that more aggressive reduction of words to common forms significantly changes the relationship between the unigram and bigram components of the TF-IDF vocabulary.

3.4. Overview of the Applied Models and Common Training Workflow

All applied models share a common training and evaluation workflow, implemented through the train_and_evaluate() function (src/modeling.py module), ensuring methodological consistency of the comparison. The processing flow for each model×variant combination is as follows

1. **GridSearchCV** search over the selected hyperparameter grid, exclusively on the training set, using 5-fold cross-validation and the F1 metric as the criterion for selecting the best hyperparameter combination. The test set is not used at this step.
2. **Final model** – the hyperparameter combination with the best CV F1 result is then trained on the entire training set.
3. **Evaluation** – the resulting model is applied to the test set for the first time, where accuracy, precision, recall, and F1 metrics are calculated, as well as the confusion matrix.
4. **Saving results** – the trained model (.joblib), metrics and best hyperparameters (.json), and confusion matrix (.png) are saved to disk for subsequent consolidated analysis in Chapter 4.

```
grid_search = GridSearchCV(
    model, param_grid, cv=cv_folds, scoring="f1", n_jobs=-1
)
grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
```

Source Code Segment GridSearchCV Search

For each of the six models considered, the hyperparameter grid was deliberately targeted rather than an exhaustive search of the entire space of possible values – hyperparameters were selected based on theoretical expectations about model behavior (e.g. the expected range of a “good” regularization level) and by checking result behavior around the initial values, rather than randomly or relying exclusively on the library defaults.

- **Naive Bayes** – search for the smoothing hyperparameter alpha (Laplace/Lidstone smoothing), in the range from 0.01 to 1.0.
- **Logistic Regression** – search for the regularization parameter C and the regularization type l1_ratio (pure L1 versus pure L2), using solver="liblinear".
- **Decision Tree** – search for the maximum tree depth max_depth and minimum number of samples per leaf min_samples_leaf, to control overfitting.

- **Random Forest** – search for the number of trees n_estimators, maximum depth max_depth, number of features per split max_features, and min_samples_leaf.
- **KNN** – due to the "curse of dimensionality" in the original TF-IDF space (66,000+ features), KNN requires prior dimensionality reduction using TruncatedSVD (chosen because, unlike PCA, it operates directly on sparse matrices without the need to center the data). Only in this reduced space is the search for the hyperparameters n_neighbors, weights, and metric performed.
- **SVM** – the linear variant (LinearSVC) was used, which is computationally much more efficient than a classical SVM with kernel functions in the case of a large number of examples and features. The search includes the regularization parameter C and the loss function loss (hinge versus squared_hinge).

3.4.1. Lite Variants of the Random Forest and KNN Models

From the results obtained by the standard GridSearchCV search (exclusively by the F1 criterion), it was observed that the Random Forest and KNN models resulted in extremely large .joblib files (up to 6.4 GB), impractical for storage and sharing, especially compared with the other models (all below 1 MB). For this reason, an additional, dedicated search was conducted for both models, whose goal was not to maximize the F1 result (this had already been achieved by the standard search), but to find a favorable compromise between the F1 result and file size.

This search was conducted in two steps – coarse, over a wide range of hyperparameters, and fine, focused on a narrower region that proved promising in the coarse step – after which Pareto analysis was used to identify the hyperparameter combination that gives the best F1 result without exceeding the target file size (~10 MB). Detailed results of this search and the obtained trade-offs are shown in Chapter 4.

3.5. Metrics and Evaluation method

Standard binary classification metrics were used to evaluate all models: accuracy, precision, recall, and the F1 measure (the harmonic mean of precision and recall), with the confusion matrix as an additional display of error types. The F1 measure was selected as the primary model ranking criterion because it represents a balanced measure that takes both precision and recall into account, which is important in sentiment analysis where both types of errors (false positives and false negatives) have equal practical significance.

Since the main results are based on a single, fixed training/test split (for execution speed and direct comparability of all 24 model×variant combinations), two additional statistical checks were conducted to assess the reliability and significance of the obtained differences among the best models:

McNemar test. An exact binomial test that, for two classifiers applied to the same test set, considers only examples on which the models disagree (discordant pairs), and based on their ratio assesses whether the difference in the errors of the two models is statistically significant ($p < 0.05$) or can be attributed to chance.

```
n10 = np.sum(correct_a & ~correct_b)
n01 = np.sum(~correct_a & correct_b)
result = binomtest(min(n10, n01), n10 + n01, p=0.5)
```

Source Code Segment McNemar Test

Bootstrap confidence interval. For the best-ranked models, the test set was resampled (with replacement) over 2,000 iterations, and the F1 measure was calculated for each resulting sample. Based on the resulting distribution of F1 values, a 95% confidence interval was constructed, estimating the stability of the point F1 estimate with respect to changes in the test-set sample.

Both checks make it possible to correctly interpret the difference between two closely ranked models (e.g. Logistic Regression and SVM, whose F1 difference is only 0.0005) as a real difference in performance or as noise arising from a single, fixed train/test split.

3.6. Limitations of the Experiment

The conducted experiment rests on several methodological limitations, explicitly stated for the correct interpretation of the results in Chapter 4:

- All main results were obtained on one fixed train/test split (random_state=1), rather than through repeated cross-validation over the entire dataset. This decision was made for practical feasibility (individual models, such as Random Forest, required up to 15 hours of training per variant) and direct comparability of all 24 model×variant combinations, which share an identical test set.
- A small difference in F1 result (on the order of a few tenths of a percent) should not be interpreted as evidence of a statistically significant difference between models unless that significance is confirmed by the McNemar test.
- The hyperparameter grids for the GridSearchCV search are not exhaustive, but deliberately targeted at an area where, based on theoretical considerations and preliminary checks, good results are expected – therefore, it cannot be claimed that the absolutely optimal hyperparameters were found for each model.
- Model training time was measured on one specific hardware setup and depends on the number of processor cores (n_jobs=-1), so it is not treated as an absolutely comparable metric between models, but as an indicative measure of practical usability (discussed in more detail in Chapter 4).

4. Results and Analysis

This chapter presents the results of the conducted experiment and their analysis. The goal of the analysis is to compare different machine learning algorithms based on the obtained results and to examine the influence of four text preprocessing variants on the success of movie review classification.

In addition to the classification results themselves, the results of additional statistical checks will be considered, as well as practical aspects of model application, including the size of the trained models and the time required to train them. In this way, the results will be considered not only through the achieved metric values, but also through the limitations and conditions under which those results were obtained.

4.1. Overview of All Results

Before proceeding to a detailed analysis of individual models, this section provides a consolidated overview of the results of all 32 tested model×variant combinations. Eight models were included in the experiment, where in addition to the six basic models, lite variants of the Random Forest and KNN models were included, and each was tested on four text preprocessing variants. For clarity, the F1 measure for all 32 combinations is shown for the initial comparison because it represents the primary criterion used when selecting model hyperparameters.

Table Comparison of preprocessing variants for each model

model_name	base	stem	lem	lanc	best_variant
naive_bayes	0.8875	0.8846	0.8813	0.8824	base
logistic_regression	0.9123	0.9112	0.9096	0.9091	base
decision_tree	0.7613	0.7662	0.7611	0.7618	stem
random_forest	0.8848	0.8838	0.8801	0.8786	base
lite_random_forest	0.8686	0.8685	0.8666	0.8673	base
knn	0.8200	0.8150	0.8090	0.8118	base
lite_knn	0.8324	0.8292	0.8208	0.8245	base
svm	0.9118	0.9109	0.9102	0.9097	base

The best result in the entire experiment was achieved by the combination of the Logistic Regression model and the baseline preprocessing variant, with an F1 measure of 0.9123, immediately followed by the combination of the SVM model and the baseline variant, with an F1 measure of 0.9118. The eight best combinations have very small differences in F1 measure, only 0.0032, while the differences in other evaluation metrics, such as accuracy, precision, and recall, are also relatively small. This indicates very similar behavior among the most successful combinations of models and preprocessing

variants, so this group of results can be regarded as practically equivalent, which will be further discussed below.

When interpreting these differences, it is necessary to take into account the limitation of the experiment based on a single fixed split into training and test sets.

Table Top 10 overall models

rank	model_name	variant	f1	accuracy	precision	recall	size_mb
1	logistic_regression	base	0.9123	0.9110	0.9025	0.9222	0.50
2	svm	base	0.9118	0.9107	0.9034	0.9204	0.50
3	logistic_regression	stem	0.9112	0.9103	0.9046	0.9180	0.51
4	svm	stem	0.9109	0.9097	0.9021	0.9198	0.51
5	svm	lem	0.9102	0.9089	0.9007	0.9200	0.51
6	svm	lanc	0.9097	0.9086	0.9030	0.9164	0.52
7	logistic_regression	lem	0.9096	0.9084	0.9012	0.9182	0.51
8	logistic_regression	lanc	0.9091	0.9081	0.9026	0.9158	0.52
9	naive_bayes	base	0.8875	0.8858	0.8773	0.8979	1.98
10	random_forest	base	0.8848	0.8816	0.8645	0.9062	205.58

After the consolidated overview of the results, the results are analyzed individually for each of the applied models below. For each model, the results for all four text preprocessing variants are shown, together with the corresponding evaluation metrics and selected hyperparameters, followed by a brief analysis of the obtained results, computational efficiency, and specific characteristics of the particular approach.

4.2. Naive Bayes

Naive Bayes, despite having the simplest theoretical basis of all tested models, achieved surprisingly good results. Although it does not consider complex dependencies between terms, its simplicity proved sufficient in this case to achieve a relatively high level of classification success. Nevertheless, Naive Bayes achieved the third-best result in the entire experiment, immediately behind the two linear models — Logistic Regression and SVM.

Table Naive Bayes Results

Variant	Accuracy	Precision	Recall	F1	Best alpha
Baseline	0,8858	0,8773	0,8979	0,8875	0,2
Stemming	0,8830	0,8759	0,8935	0,8846	0,125
Lemmatization	0,8794	0,8705	0,8925	0,8813	0,225
Lancaster	0,8804	0,8709	0,8943	0,8824	0,275

The Baseline variant gives a slightly, but consistently, better result than the remaining three normalization variants, as explained in Section 4.8. The key advantage of Naive Bayes lies in its exceptional computational efficiency: the GridSearchCV search finishes in only a few seconds (on the order of 8–15 s per variant), and the size of the saved model is only about 2 MB – the model is simultaneously third in the overall ranking (F1 = 0.8875) and by far the fastest to train among all models that are not narrowly linear. The main disadvantage of the model stems precisely from its theoretical assumption: conditional independence of terms given the class is almost certainly not true for natural language, which statistically significantly separates it (Section 4.8) from the group of linear models (Logistic Regression, SVM), although the difference in absolute F1 is only two percentage points.

4.3. Logistic Regression

Logistic Regression achieved the best result in the entire experiment, with the baseline variant achieving an F1 measure of 0.9123. In addition to high classification quality, the model also proved very efficient in terms of training time and saved model size, which makes it particularly significant for further analysis and comparison with other approaches.

Table Logistic Regression Results

Variant	Accuracy	Precision	Recall	F1	Best C
Baseline	0,9110	0,9025	0,9222	0,9123	6,0
Stemming	0,9103	0,9046	0,9180	0,9112	5,0
Lemmatization	0,9084	0,9012	0,9182	0,9096	7,0
Lancaster	0,9081	0,9026	0,9158	0,9091	6,0

For all four variants, the best hyperparameter combination involved pure L2 (l1_ratio=0.0) regularization, indicating that the sentiment signal in the data is not concentrated in a small number

of exceptionally strong terms, but is distributed across thousands of moderately scored terms – a finding also confirmed by a separate statistical analysis conducted during vectorization.

Analysis of the model coefficients confirmed that the model actually uses the preserved negations: terms such as “not disappoint”, “not recommend”, and “not funny” are among the twenty most influential terms for the positive and negative class, respectively, thereby verifying the correctness of the decision to retain negation words in the stop-word list.

Table Most Influential Terms (base variant)

Positive sentiment term	Positive sentiment strength	Negative sentiment term	Negative sentiment strength
great.	+10.979	worst	-15.315
excellent.	+10.967	awful	-12.851
perfect	+8.530	bad	-11.145
amazing	+7.989	boring	-10.889
best	+7.684	waste	-10.564
wonderful	+7.669	terrible	-9.923
hilarious	+7.571	poor	-9.374
loved	+7.194	nothing	-8.333
brilliant	+7.017	not funny	-8.123
favorite	+6.954	disappointment	-8.036

Logistic Regression represents the best compromise in the entire work: it achieves the best F1 result, trains in only 2–3 minutes (the fastest after Naive Bayes and SVM), and the size of the saved model does not exceed 1 MB. The model is also directly interpretable – the direction and strength of the influence of each term can be read directly from its coefficient, unlike ensemble methods. The main disadvantage of the model is its linear nature, which makes it theoretically less expressive than nonlinear models (Random Forest, KNN) – nevertheless, in this experiment that additional expressiveness did not materialize in a better result for the other models, suggesting that the sentiment analysis problem over a TF-IDF representation is predominantly linearly separable.

4.4. Decision Tree

Decision Tree is, unlike all other models, the only one for which the Baseline variant did not give the best result. This model also showed considerably weaker overall performance compared with the other

approaches tested, making it an interesting example for analysing the influence of algorithm choice on the final result.

Table Decision Tree Results

Variant	Accuracy	Precision	Recall	F1	Best max_depth / min_samples_leaf
Baseline	0,7333	0,6910	0,8475	0,7613	10 / 2
Stemming	0,7444	0,7082	0,8344	0,7662	10 / 1
Lemmatization	0,7305	0,6854	0,8557	0,7611	8 / 2
Lancaster	0,7392	0,7033	0,8310	0,7618	12 / 3

Decision Tree is by far the weakest model in the entire experiment by F1 measure (all four variants below 0.77), with a pronounced difference between precision (~0.69–0.71) and recall (~0.83–0.86) – the model is considerably more prone to falsely classifying negative reviews as positive. Analysis of feature_importances_ reveals that an individual tree, despite a vocabulary of over 60,000 terms, uses only a few hundred terms in its construction (e.g. 209 of 66,682 in the stem variant), indicating extremely shallow and narrow utilization of the available TF-IDF space. The main advantage of the model is direct interpretability (it is possible to follow the exact sequence of decisions that led to each prediction) and extremely small file size (tens of KB) – however, the model primarily serves as a basis for comparison with Random Forest (Section 4.5), where it is shown that ensembling a large number of such trees significantly improves the result.

4.5. Random Forest

The Random Forest model was analyzed in two variants, differing according to the optimization objective. The first, heavy variant, represents the result of the standard GridSearchCV search, in which the F1 measure was used as the sole criterion for hyperparameter selection. This approach produces a model with the best result within the given hyperparameter grid, but without taking its size and practicality for storage and use into account. Therefore, after analyzing the heavy variant, an additional lite search was conducted, whose goal was to find a favorable compromise between the F1 result and model size.

4.5.1. Heavy Variant

First, the results of the standard, heavy Random Forest variant, obtained through a GridSearchCV search aimed exclusively at maximizing the F1 measure, were analyzed.

Table Random Forest Heavy Results

Variant	F1	File size	Best hyperparameter combination
Baseline	0,8848	210,5 MB	depth=500, leaf=2, n=1500
Stemming	0,8838	6,4 GB	depth=500, leaf=1, n=2500
Lemmatization	0,8801	202 MB	depth=400, leaf=2, n=1500
Lancaster	0,8786	199 MB	depth=500, leaf=2, n=1500

The extremely large file size in the stem variant (6.4 GB) is a consequence of GridSearchCV, guided exclusively by the F1 criterion, selecting an extreme combination (min_samples_leaf=1, depth 500, 2,500 trees) for that variant, without considering the practical consequences for model size. At the same time, in all preprocessing variants, the value log2 was selected as the best choice for the max_features parameter, indicating that limiting the number of features considered at each tree split was more favorable than the alternative tested values within the used hyperparameter grid.

4.5.2. Lite Variant

Given the impractically large model sizes obtained by the standard search, an additional dedicated search described in Section 3.4.1 was conducted for each preprocessing variant, with the aim of finding a more favorable relationship between F1 result and model size. The results of this search are shown below.

Table Random Forest Lite vs Heavy

Variant	F1 (heavy)	F1 (lite)	Δ F1	Size (heavy)	Size (lite)	Reduction factor
Baseline	0,8848	0,8686	−0,0162 (−1,83%)	210,5 MB	8,19 MB	~26×
Stemming	0,8838	0,8685	−0,0153 (−1,73%)	6.400 MB	8,91 MB	~718×
Lemmatization	0,8801	0,8666	−0,0135 (−1,53%)	202 MB	9,60 MB	~21×
Lancaster	0,8786	0,8673	−0,0113 (−1,29%)	199 MB	9,60 MB	~21×

Random Forest, in its original form, does not justify its substantially higher computational and memory cost compared with linear models – in no variant does it reach the F1 result of Logistic

Regression or SVM, despite training taking up to 15 hours per variant (as opposed to several minutes for linear models). Nevertheless, the lite variant represents a very practical alternative: with a consistent and relatively small loss in F1 measure (in the range of 1.3–1.8%), the file size was reduced by 21 to as much as 718 times, depending on how "inflated" the result of the original GridSearchCV search was for the particular variant. This represents the key practical advantage of the lite approach: it does not seek a better model at any cost, but a reasonable compromise between quality and usability.

The relationship between the F1 measure and model size is additionally shown by the Pareto line, which connects combinations representing the most favorable compromise between classification quality and memory requirements. The graph shows how F1 gradually improves as model size increases, but after a certain point, further increases in size bring progressively smaller gains in performance. This analysis served as the basis for selecting the final parameters of the lite variant.

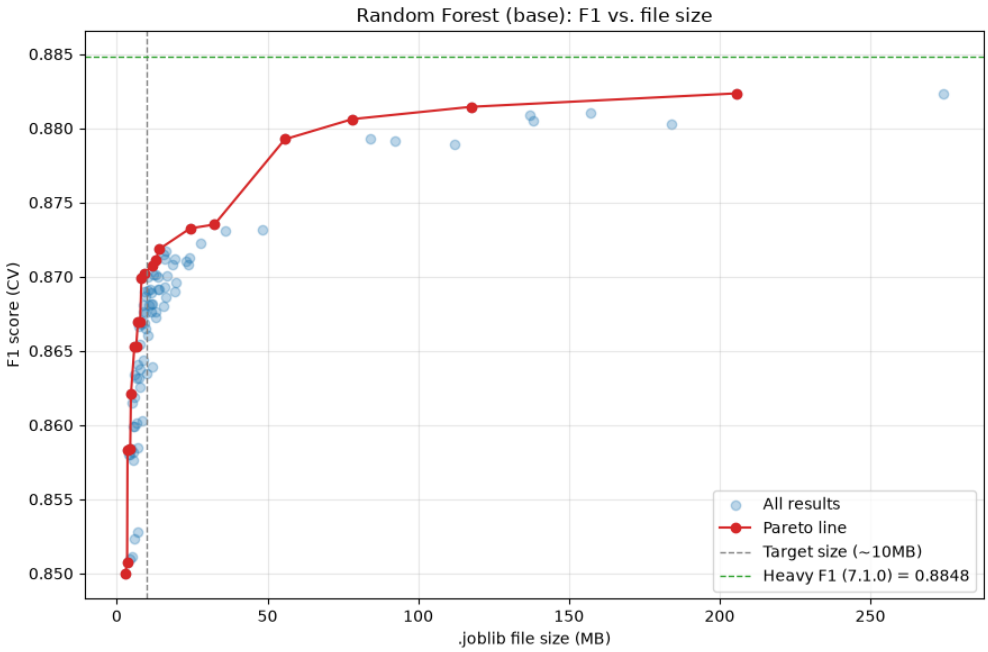


Figure F1 versus file size (Pareto line) for Random Forest lite (base)

4.6. K-Nearest Neighbors (KNN)

The KNN model was analyzed in two variants due to the specifics of working in a high-dimensional TF-IDF space. First, the heavy variant with a larger number of components obtained using TruncatedSVD was tested, followed by the lite variant, aimed at finding a smaller number of components that provides a favorable relationship between F1 result and model size.

The cumulative explained variance in both variants increases almost linearly with the number of SVD components, without a pronounced saturation point. This curve shape indicates that additional components continuously capture an increasing portion of the variability in the TF-IDF space, making it impossible to identify a natural optimal boundary solely on the basis of explained variance.

4.6.1. Heavy Variant

KNN requires prior dimensionality reduction of the TF-IDF space (66,000+ features) using TruncatedSVD, since in the original space the "curse of dimensionality" makes the concept of "closeness" meaningless. The number of SVD components was selected based on cumulative explained variance, where the cumulative curve shows no clear saturation point even up to 5,000 components (the largest practically feasible value on the available hardware), so this value was adopted as final.

Table KNN heavy

Variant	Explained variance (5000 comp.)	F1	File size
Baseline	56,9%	0,8200	~1,5 GB
Stemming	57,9%	0,8150	~1,5 GB
Lemmatization	57,0%	0,8090	~1,5 GB
Lancaster	57,8%	0,8118	~1,5 GB

The heavy variant shows that a large number of SVD components does not bring a corresponding improvement in classification performance. F1 results remain in a relatively narrow range from 0.8090 to 0.8200, while model size remains around 1.5 GB for all four variants. These results indicate that using 5,000 components carries a significant memory cost, without a proportional gain in classification quality, which motivated testing a more compact lite variant.

4.6.2. Lite Variant

The dedicated search for the lite version showed a counterintuitive but theoretically explainable result: the F1 measure does not increase monotonically with the number of SVD components, unlike cumulative explained variance, which increases monotonically throughout the tested range. The reason is that TruncatedSVD ranks components by the amount of explained variance, not by their relevance to sentiment. While a smaller number of the first components may contain a significant portion of information relevant for distinguishing positive and negative reviews, adding a large number of subsequent components also introduces information that is not necessarily related to sentiment, such as thematic and genre characteristics of the text. In cosine-based KNN, such additional components can affect the measurement of similarity between documents and thereby reduce classification quality.

An additional check showed that switching from float64 to float32, and then to float16 storage format, does not measurably affect the F1 result (differences at the fourth decimal place), while the file size decreases almost linearly – float16 was therefore adopted for all further lite searches and the final model, without any impact on quality.

Table KNN lite vs heavy

Variant	F1 (heavy)	F1 (lite)	Δ F1	Components (heavy → lite)	Size (heavy → lite)	Reduction factor
Baseline	0,8200	0,8324	+0,0124 (+1,51%)	5000 → 10	1,5 GB → ~1,0 MB	~1500×
Stemming	0,8150	0,8292	+0,0142 (+1,74%)	5000 → 15	1,5 GB → ~1,4 MB	~1050×
Lemmatization	0,8090	0,8208	+0,0118 (+1,46%)	5000 → 15	1,5 GB → ~1,4 MB	~1050×
Lancaster	0,8118	0,8245	+0,0127 (+1,56%)	5000 → 15	1,5 GB → ~1,4 MB	~1050

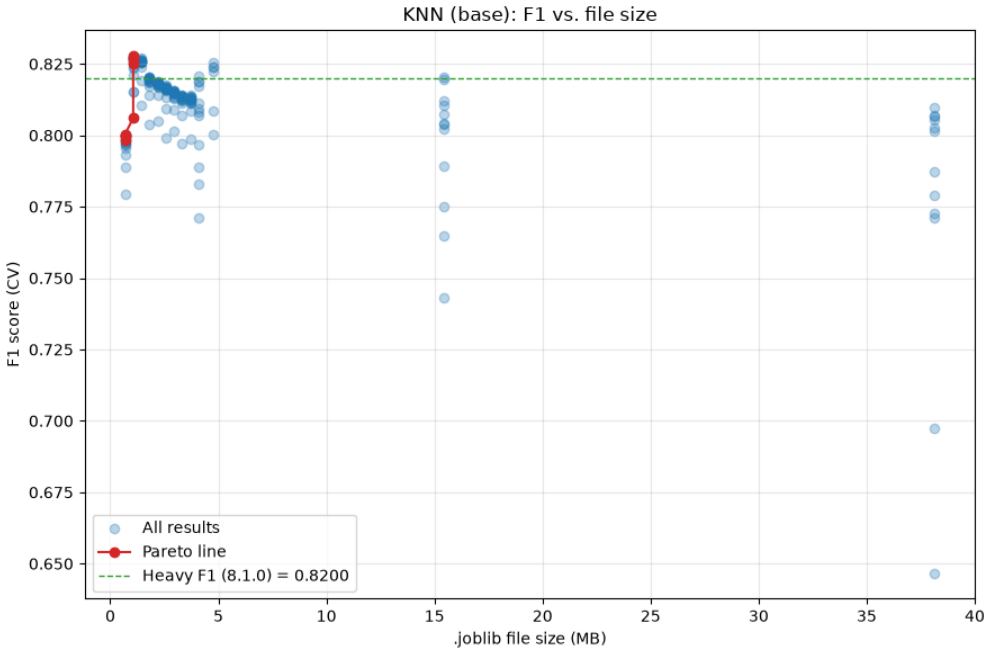


Figure F1 versus file size (Pareto line) for KNN lite (base)

KNN represents the weakest of the six basic models in its original form (F1 in the range 0.809–0.820), with an extremely high memory cost (~1.5 GB per variant), a consequence of storing the entire reduced training matrix. The results of the lite search, however, represent an important methodological finding of the work: unlike Random Forest, where the lite version inevitably sacrifices some F1 result for smaller size, the KNN lite version simultaneously achieves a better F1 result (by 1.2–1.7 percentage points) and a smaller file size (by more than 1,000 times). This showed that the original method of selecting the number of components (based on explained variance) was methodologically misdirected

for this specific classification task – direct optimization of the target metric (F1) proved not only more efficient, but also fundamentally better.

4.7. Support Vector Machine (SVM)

Compared with the previously considered models, SVM represents another linear classification approach, particularly suitable for working with high-dimensional TF-IDF representations. The results for all four preprocessing variants are shown below, together with an analysis of the selected hyperparameters and a comparison of the achieved metrics.

Special attention was paid to comparing SVM with the Logistic Regression model, since both approaches build a linear separating boundary in the same TF-IDF space and achieve very similar results. In addition to classification quality, practical aspects of model application were considered, primarily training time and saved model size, so that the results could be viewed not only through the F1 measure, but also in the context of computational efficiency and usability.

Table SVM Results

Variant	Accuracy	Precision	Recall	F1	Best combination
Baseline	0,9107	0,9034	0,9204	0,9118	C=0,4, loss=squared_hinge
Stemming	0,9097	0,9021	0,9198	0,9109	C=0,3, loss=squared_hinge
Lemmatization	0,9089	0,9007	0,9200	0,9102	C=0,3, loss=squared_hinge
Lancaster	0,9086	0,9030	0,9164	0,9097	C=0,7, loss=squared_hinge

SVM (in the linear LinearSVC variant) achieves results practically equal to those of Logistic Regression (difference at the second and third decimal places, statistically unconfirmed – Section 4.8), with an almost identical small file size (below 1 MB) and very short training time (about 55–80 seconds per variant, even faster than Logistic Regression). Comparison of the SVM and Logistic Regression coefficients showed that both models identify almost the same set of terms as the most influential for sentiment, further confirming that the problem over a TF-IDF representation is well suited to linear separation methods. The main practical difference compared with Logistic Regression is that LinearSVC does not provide a direct probability estimate, while Logistic Regression enables this through predict_proba, which is significant for the GUI application.

Table Most Influential Terms (base variant)

Positive sentiment term	Positive sentiment strength	Negative sentiment term	Negative sentiment strength
excellent.	+3.198	worst	-4.453
great.	+3.060	awful	-3.783
perfect	+2.379	bad	-3.287
amazing	+2.290	boring	-3.195
best	+2.192	waste	-3.005
hilarious	+2.179	terrible	-2.887
wonderful	+2.168	poor	-2.833
today	+2.118	disappointment	-2.505
loved	+2.083	not worth	-2.477
fun	+2.080	disappointing	-2.449

4.8. Evaluation and Analysis of Results

After the individual analysis of all tested models, the results are considered in aggregate below, with the aim of examining the influence of different factors on the final system performance and identifying general patterns observed during the experiment.

4.8.1. Influence of the Text Preprocessing Variant

The consolidated comparison of F1 results by preprocessing variant, for all eight models, is shown in the following graph. The graph enables a direct comparison of the influence of four approaches to text normalization on the performance of different classification models. In this way, it is possible to determine whether the observed effects of individual variants are repeated across different algorithms or depend on the specifics of the model itself.

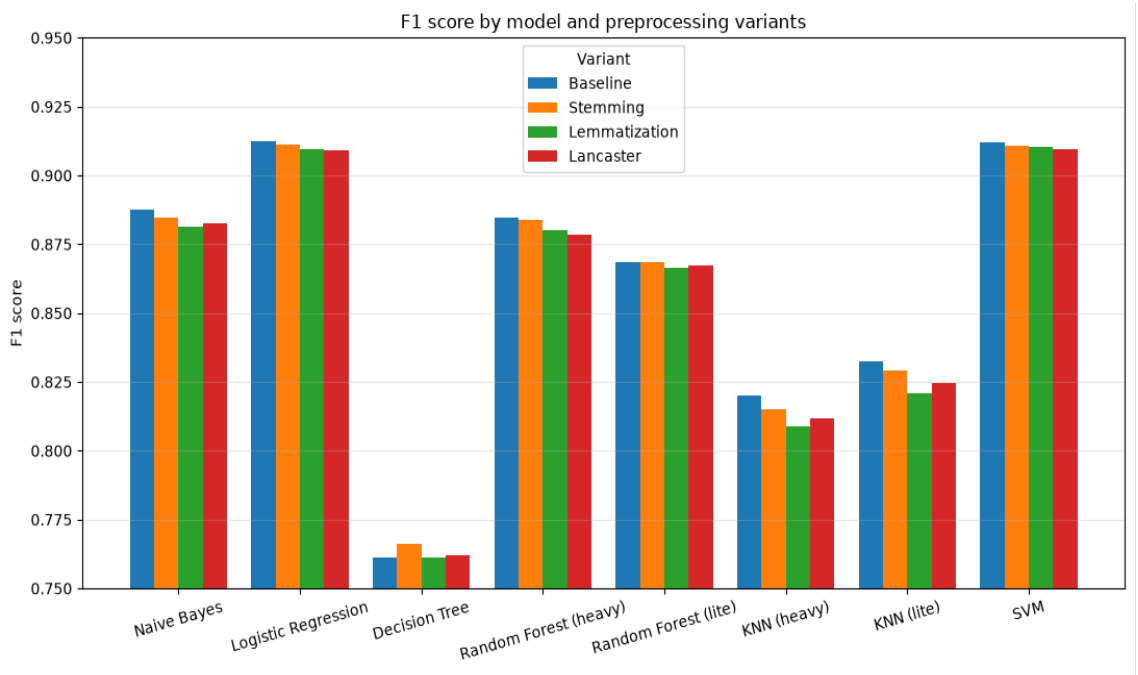


Figure Comparison of preprocessing variants within each model

Contrary to the initial hypothesis (that normalization of word forms would improve performance by reducing vocabulary size and aggregating similar word forms into a common token), the results show the opposite trend: the Baseline variant gives the best or tied result for 7 of 8 models; the only exception is Decision Tree, where the stem variant is slightly ahead (difference 0.0049 F1). Possible causes of this finding, also discussed in detail in 10_evaluation.ipynb, include:

- bigrams (ngram_range=(1,2)) partially take over the role that normalization would otherwise have, making the reduction in granularity through stemming/lemmatization less useful than it would be for a purely unigram approach;
- normalization reduces the granularity of the representation (e.g. “loved”/“loving”/“love” become an identical token), causing the model to lose the ability to learn different weights for different forms of the same word;
- linear models with L2 regularization make good use of a large, “raw” vocabulary with a distributed (moderate) signal, supporting the baseline variant.

Lancaster (more aggressive stemmer) did not outperform Snowball stemming in any case, refuting the initial hypothesis that more aggressive suffix removal would provide an additional improvement.

4.8.2. Statistical Significance and Reliability of Results

Since all main results were obtained on one fixed train/test split, differences in model performance may to some extent depend on the specific choice of examples included in the training and test sets. Therefore, in addition to the basic evaluation, additional statistical checks were needed to assess whether the observed differences between the best models were sufficiently pronounced to be considered statistically significant rather than a consequence of the randomness of the specific data

split. For this purpose, two additional approaches were applied: the McNemar test and the bootstrap confidence interval.

McNemar test

Comparison of the Logistic Regression and SVM models (baseline variant) produced $p = 0.7877$, meaning that the difference in F1 measure (0.0005) is not statistically significant – the two models should be treated as practically equivalent. In contrast, comparison of Logistic Regression and Naive Bayes models (baseline variant) gives $p < 0.0001$, confirming that the difference between the group of linear models and Naive Bayes is statistically significant.

Bootstrap confidence interval

Bootstrap analysis was used to assess the stability of F1 results with respect to changes in the test-set sample. Based on 2,000 bootstrap resamplings, a 95% confidence interval [0.9060; 0.9178] was obtained for Logistic Regression baseline (F1 = 0.9123) and [0.9055; 0.9175] for SVM baseline (F1 = 0.9118). Their intervals almost completely overlap, which is consistent with the McNemar test that did not show a statistically significant difference.

For Naive Bayes baseline, an interval [0.8812; 0.8938] was obtained around the F1 value 0.8875, which does not overlap with the intervals of the linear models. This additionally indicates a stable difference in performance level. Bootstrap does not directly test the statistical significance of the difference, but shows the stability of the F1 estimate itself.

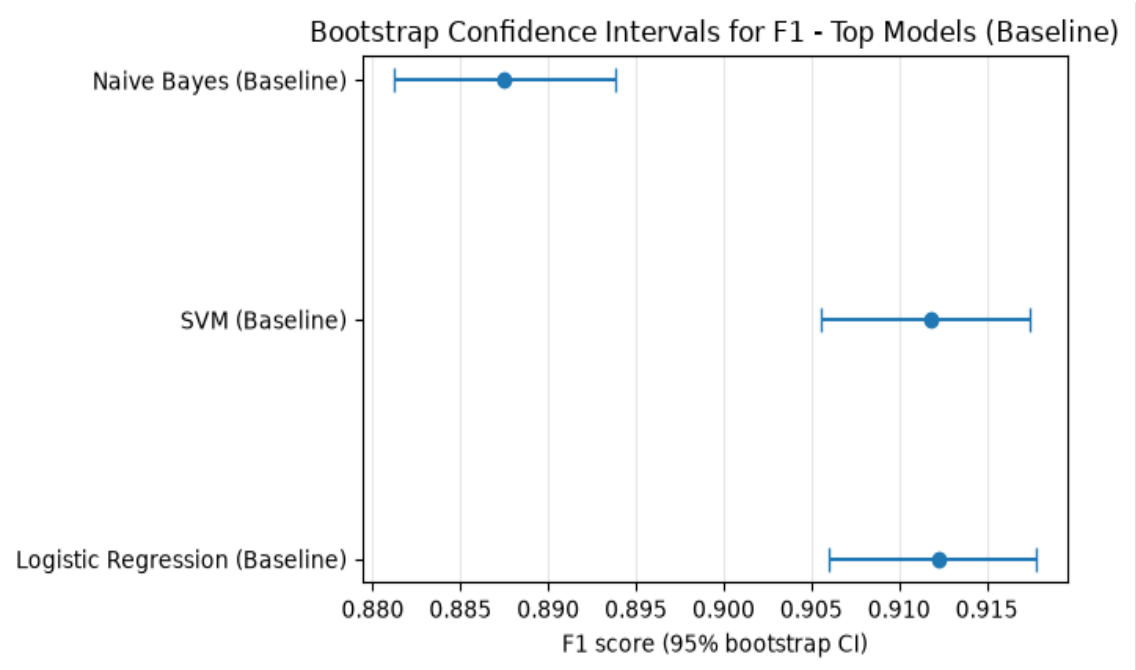


Figure Bootstrap confidence interval

The Logistic Regression and SVM intervals almost completely overlap, consistent with the finding of the McNemar test, while the Naive Bayes interval does not overlap with them, thereby confirming a real difference in performance.

Table Bootstrap Confidence Interval (95%, 2,000 samples)

Model	Point F1 estimate	95% confidence interval
Logistic Regression (baseline)	0,9123	[0,9060 ; 0,9178]
SVM (baseline)	0,9118	[0,9055 ; 0,9175]
Naive Bayes (baseline)	0,8875	[0,8812 ; 0,8938]

4.8.3. Training Time and Model Size

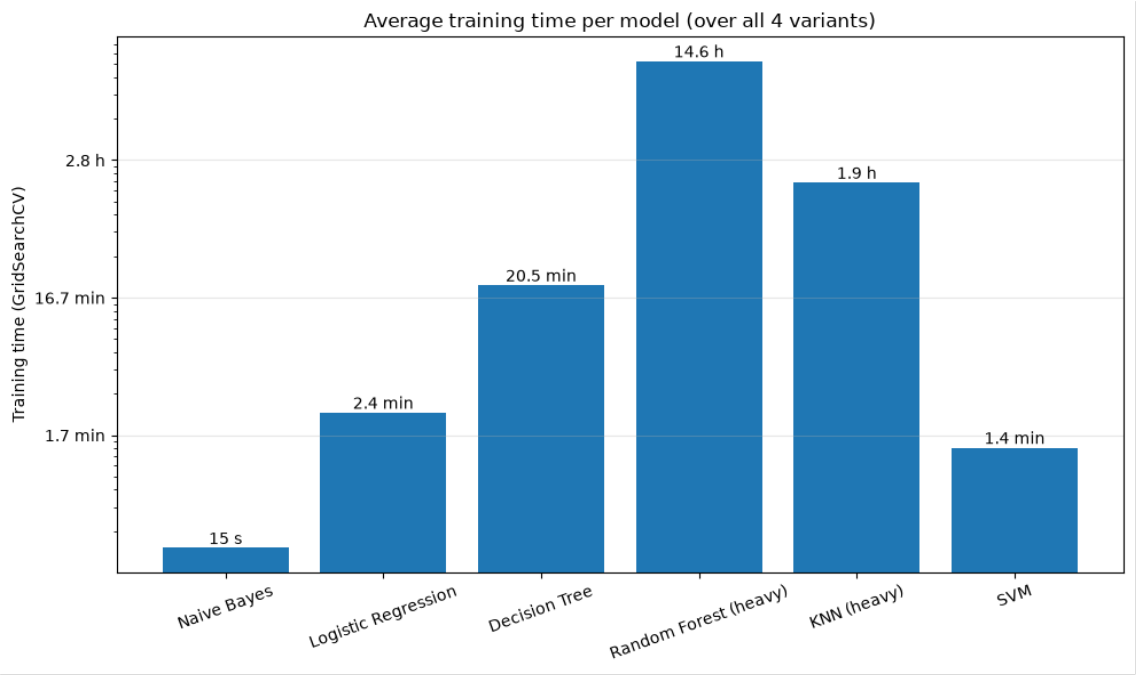


Figure Average training time per model (logarithmic scale)

The ordering of models by training speed, averaged over all four preprocessing variants, shows a clear difference between individual approaches: Naive Bayes requires only a few seconds, SVM about 1.5 minutes, Logistic Regression about 2.5 minutes, while Decision Tree requires about 20 minutes. For more complex models, the time increases further, with KNN heavy requiring about 2 hours, and Random Forest heavy as much as about 15 hours per variant. These results show that an increase in computational complexity was not accompanied by a proportional improvement in F1 results. It is particularly significant that Logistic Regression, which achieved the highest F1 result, also belonged to the group of models with relatively short training times.

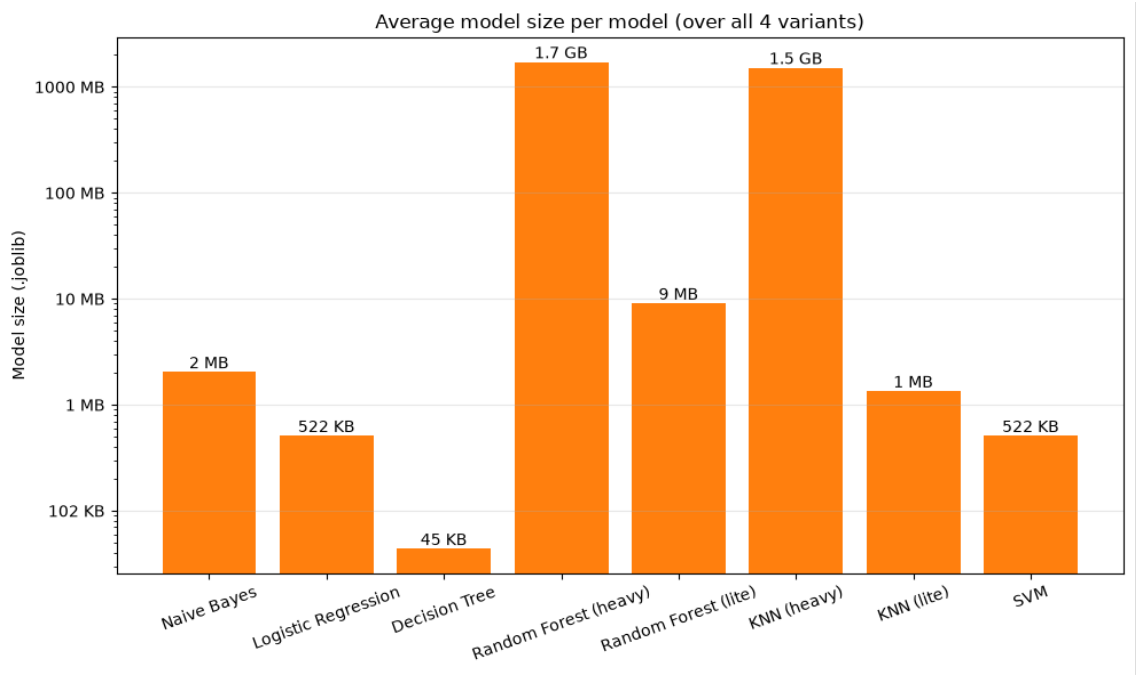


Figure Average file size per model (logarithmic scale)

A similar relationship is observed in the size of the saved models. Decision Tree, Logistic Regression, and SVM require less than 1 MB, making them highly suitable for practical application and distribution. In contrast, the heavy variants of Random Forest occupy about 200 MB to as much as 6.4 GB, while KNN heavy models require approximately 1.5 GB per variant. This large difference further demonstrates the importance of considering not only classification performance but also the practical resources required to train and use the models.

4.8.4. Pareto Analysis (F1 versus “Cost”)

Pareto analysis makes it possible to consider results not only through the achieved F1, but also in relation to the practical cost of the model, measured by the size of the saved model and training time. In this way, combinations can be identified where an increase in resources also brings an additional improvement in performance.

When model size is considered as the cost, the Pareto front begins with Decision Tree variants, where F1 values up to 0.7662 are achieved with very small models. This is followed by a large jump with the SVM baseline variant, which reaches F1 = 0.9118 at about 0.5 MB, while Logistic Regression baseline further increases F1 to 0.9123 with practically the same model size. After that, larger models do not bring further improvement in F1 results.

When training time is considered as the cost, Naive Bayes represents the starting point with very short training time, while the largest jump in performance occurs when moving to SVM. SVM stem stands out with F1 = 0.9109 for approximately 59 seconds, while Logistic Regression baseline reaches the highest F1 of 0.9123 in about 162 seconds. Further increases in training time for more complex models in this experiment do not bring a comparable improvement in results.

Table Pareto Analysis by Model Size

model_name	variant	f1	size_mb
decision_tree	lem	0.761147	0.022210
decision_tree	base	0.761303	0.040368
decision_tree	stem	0.766165	0.043878
svm	base	0.911823	0.495112
logistic_regression	base	0.912253	0.495250

Table Pareto Analysis by Model Training Time

model_name	variant	f1	train_time_seconds
naive_bayes	lem	0.881349	10.782996
naive_bayes	stem	0.884623	13.401442
naive_bayes	base	0.887499	17.619178
svm	stem	0.910864	59.075599
svm	base	0.911823	95.588396
logistic_regression	base	0.912253	161.816609

4.8.5. Precision and Recall Asymmetry

In 29 of the 32 tested combinations, recall is higher than precision, indicating a slight tendency of the models to incorrectly classify negative reviews as positive rather than the reverse. The exception is heavy KNN, for which this relationship does not occur in the baseline, stemmed, and Lancaster stem variants. This suggests that the cause is more likely to lie in the structure of the data itself than in the specifics of a single algorithm.

A possible explanation is the fact that negative reviews may contain pronounced positive elements, such as praise for actors, music, or individual scenes, even though the overall opinion of the film is negative. In the Logistic Regression baseline variant, a falsely positive classified example was found in which an actress is described with the expressions “lovely auburn-haired actress” and “exuded intelligence”, while the negative attitude toward the film itself is formulated much more briefly. The model classified this negative review as positive with a probability of 0.7650, illustrating how strong positive signals within a negative review can influence the final classification.

5. Conclusion

The goal of this work was to conduct sentiment analysis of movie reviews using classical machine learning methods and to compare the performance of six different algorithms – Naive Bayes, Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, and Support Vector Machine – over the same TF-IDF text representation, while examining the influence of four different preprocessing techniques (without normalization, stemming, lemmatization, and aggressive stemming).

The results of the conducted experiment showed that the linear models, Logistic Regression and SVM, applied to the unprocessed (baseline) text, achieved the best and statistically indistinguishable results (F1 = 0.9123 and F1 = 0.9118, respectively), confirmed by the McNemar test (p = 0.7877) and overlapping bootstrap confidence intervals. In contrast, the more complex and computationally demanding models – Random Forest and KNN – failed to outperform either the linear models or the much more computationally efficient Naive Bayes, despite requiring up to 15 hours of training and up to several gigabytes of storage for the models. This confirmed that, for the problem of sentiment analysis over a TF-IDF representation of this scale, additional model complexity does not provide a corresponding return in result quality, and that the problem is largely linearly separable in the given feature space.

A particularly valuable finding of the work concerns the influence of the text preprocessing technique: contrary to the initial hypothesis, normalization of word forms (stemming, lemmatization, aggressive stemming) almost never improved the results compared with unprocessed but cleaned text – the baseline variant proved best or tied for seven of the eight models. It was also shown that more aggressive suffix removal (Lancaster stemmer) provides no advantage over moderate (Snowball) stemming, thereby refuting that initial assumption as well.

A further contribution of the work is the lite variants of the Random Forest and KNN models, developed for practical applicability. For Random Forest, the lite version achieves a reasonable compromise – with a small loss in F1 measure (1.3–1.8%), the model size was reduced by up to several hundred times. For KNN, the result is surprisingly more favorable: the lite version simultaneously achieves a better F1 result and a much smaller model size (over 1,000 times smaller), showing that the original method of selecting the number of SVD components, based solely on explained variance, was not a good optimization criterion for the specific sentiment classification task – direct optimization of the target metric proved superior.

Based on the best obtained model (Logistic Regression, baseline variant), a standalone graphical (GUI) application was also developed, packaged as an executable .exe file using PyInstaller, which allows the user to enter an arbitrary movie review in English and immediately obtain the predicted sentiment, together with the model's confidence percentage in that prediction. The application additionally shows how each of the remaining tested models (except the computationally impractical heavy versions of Random Forest and KNN) evaluates the same review, across all four preprocessing variants, thereby practically illustrating the degree of agreement or disagreement of different models on the same input.

5.1. Limitations of the Conducted Experiment

The main methodological limitation of the work concerns the fact that all main results were obtained on one fixed train/test split, rather than through repeated cross-validation over the entire dataset – a decision made primarily for practical feasibility, given that an individual Random Forest configuration required up to 15 hours of training. Although additional statistical checks (McNemar

test, bootstrap confidence intervals) were conducted to assess the reliability and significance of the obtained differences, these checks cannot fully replace results obtained through repeated experiments over multiple different data splits. An additional limitation is that the hyperparameter grids for GridSearchCV were not exhaustive, but deliberately targeted based on theoretical expectations and preliminary checks, so it cannot be claimed that absolutely optimal hyperparameters were found for each model.

5.2. Implementation Challenges

During the development of the work, a particular challenge was managing computational and memory resources for Random Forest and KNN models – the initial GridSearchCV search, guided exclusively by the F1 criterion, resulted in models up to 6.4 GB in size (Random Forest, stem variant) and consistently ~1.5 GB (KNN, all variants), which required the development of a separate, dedicated search methodology (coarse and fine search, followed by Pareto analysis) to find practically usable lite versions of the models. An additional challenge was ensuring the methodological correctness of the entire experiment – consistently separating the training and test sets throughout all stages of processing (from data splitting, through vectorization, to hyperparameter selection), so that at no point would information from the test set leak into the model training process.

5.3. Directions for Further Work

As a natural continuation of this work, it is possible to examine the application of modern approaches based on deep neural networks and transformer architectures (e.g. pretrained models such as BERT), which could potentially capture semantic and contextual relationships that the TF-IDF representation, as a purely frequency-based representation, cannot recognize – especially in cases of sarcasm, irony, or mixed sentiment within the same review. Furthermore, the results obtained in this work relate exclusively to a dataset of movie reviews in English; extending the experiment to other domains (e.g. product reviews) or to the Serbian language would represent a valuable direction for checking the generalization of the obtained conclusions. Finally, the methodological limitation related to a single, fixed train/test split could be overcome by conducting repeated (e.g. 5- or 10-fold) cross-validation over the entire dataset, using computational resources that make this practically feasible (e.g. cluster parallelization or GPU acceleration), thereby further strengthening the obtained results.

References

[1] G. Salton, C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, 1988.

[2] scikit-learn developers, scikit-learn: Machine Learning in Python — Documentation, https://scikit-learn.org/stable/documentation.html

[3] A. McCallum, K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification," AAAI-98 Workshop on Learning for Text Categorization, 1998.

[4] L. Breiman, J. Friedman, R. Olshen, C. Stone, Classification and Regression Trees, 1984.

[5] L. Breiman, "Random Forests," Machine Learning, vol. 45, 2001.

[6] C. Cortes, V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, 1995.

[7] NLTK Project, NLTK 3.x Documentation, https://www.nltk.org/